

SCS 1302 – Discrete Mathematics 1

Complete Course Study Notes

University of Colombo School of Computing (UCSC)
Based on lectures by Prof. D D Karunaratna

August 2026

Contents

1	Introduction and Course Overview	2
2	Lesson 1: Propositional Logic	3
2.1	Propositions and Logic Fundamentals	3
2.2	Logical Operators and Compound Propositions	3
2.2.1	Truth Tables for Key Operators	3
2.2.2	Terminology for $p \rightarrow q$	3
2.3	Precedence and Associativity	4
2.4	Logical Equivalence	4
2.4.1	Table of Key Logical Equivalences	4
2.5	Duality and Functional Completeness	5
2.6	Normal Forms	5
2.6.1	Finding DNF and CNF [20, 21]	5
2.7	Satisfiability and System Specifications	5
2.7.1	Example: System Specification Consistency	6
3	Lesson 2: Predicate Logic	7
3.1	Predicates and Propositional Functions	7
3.2	Domains and Truth Sets [36]	7
3.3	Quantifiers	7
3.4	Binding, Scope, and Logical Equivalences	7
3.4.1	Negation of Quantifiers [46]	7
3.5	Nested Quantifiers and Quantifier Ordering [47]	8
4	Lesson 3: Arguments and Rules of Inference	9
4.1	Validity of Arguments	9
4.2	Rules of Inference for Propositional Logic	9
4.2.1	Logical Fallacies (Invalid Argument Forms)	9
4.3	Rules of Inference for Quantified Statements	9
4.4	The Resolution Principle and Refutation Proofs [67]	10
5	Lesson 4: Introduction to Proofs	11
5.1	Terminology of Proofs	11
5.2	Methods of Proving Implications $p \rightarrow q$	11
5.3	Proof by Cases and Exhaustive Proofs	11
5.4	Existence and Uniqueness Proofs	11

6	Lesson 5: Set Theory	12
6.1	Set Definition and Notation	12
6.2	Subsets and Proper Subsets	12
6.3	Set Identities	12
6.4	Generalized Unions and Intersections [82]	12
7	Lesson 6: Functions	13
7.1	Formal Definition	13
7.2	Properties of Functions	13
7.3	Inverse Functions and Compositions	13
7.4	Special Functions	13
8	Lesson 7: Relations	14
8.1	Cartesian Products and Binary Relations	14
8.2	Properties of Relations on a Set [95]	14
8.3	Combining Relations	14
9	Lesson 8: Number Theory and Modular Arithmetic	15
9.1	Divisibility and Basic Theorems	15
9.2	Modular Arithmetic and Congruence	15
9.3	Euler's Totient Function $\varphi(n)$	15
9.4	Residue Systems	15
9.5	The Euclidean and Extended Euclidean Algorithms	15
10	Revision Summary and Quick Reference	16
10.1	Logical Precedence	16
10.2	Logical Operator vs. Set Operator Translation	16
10.3	Proof Strategy Selector	16

1 Introduction and Course Overview

This document serves as a comprehensive, rigorous study guide for the course **SCS 1302 – Discrete Mathematics 1** (and SCS 1204 – Discrete Mathematics I) taught at the **University of Colombo School of Computing (UCSC)** by **Prof. D D Karunaratna** [1]. The primary textbook for this course is *Discrete Mathematics and Its Applications* by Kenneth H. Rosen (7th Edition), covering Chapters 1, 2, 4, and 9 [1].

The course spans eight core modules:

1. Propositional Logic
2. Predicate Logic
3. Arguments and Rules of Inference
4. Introduction to Proofs
5. Set Theory
6. Functions
7. Relations
8. Number Theory and Modular Arithmetic

2 Lesson 1: Propositional Logic

2.1 Propositions and Logic Fundamentals

A **proposition** is a declarative sentence (a sentence that declares a fact) that is either true (denoted by T or 1) or false (denoted by F or 0), but not both [2]. The true or false value assigned is called its **truth value** [2]. The systematic study of propositions is called the **propositional calculus** or **propositional logic**, first developed by Aristotle over 2300 years ago [2].

An **atomic proposition** (or primitive proposition) is a statement that cannot be broken down into smaller statements [3]. For example:

- "2 is a prime number" (Truth value: T) [3]
- "6 is 3" (Truth value: T) [3]
- "3 is 6" (Truth value: F) [3]

Propositional Variables: Letters such as p, q, r, s are used to denote atomic propositions [3]. These variables can be assigned T (1) or F (0) [3]. A **formula** is a proposition where all constituent propositions are replaced with propositional variables [8].

2.2 Logical Operators and Compound Propositions

Compound propositions are formed by combining existing propositions using logical operators.

1. **Negation:** Let p be a proposition. The negation of p , denoted by $\neg p$ (or $\sim p$), is the statement "It is not the case that p ." Negation is a *unary* operator [4].
2. **Conjunction (AND):** The conjunction $p \wedge q$ is true when both p and q are true, and false otherwise.
3. **Disjunction (OR):** The disjunction $p \vee q$ is false when both p and q are false, and true otherwise.
4. **Exclusive Disjunction (XOR):** The exclusive or of p and q , denoted by $p \oplus q$, is true when exactly one of p and q is true, and false otherwise [5].
5. **Conditional Statement (Implication):** Let p and q be propositions. The conditional statement $p \rightarrow q$ is the proposition "if p , then q ." It is false *only* when p is true and q is false, and true otherwise [5]. Here, p is called the **hypothesis** (antecedent/premise) and q is the **conclusion** (consequence) [5].
6. **Bi-conditional Statement (Bi-implication):** The statement $p \leftrightarrow q$ is the proposition " p if and only if q ." It is true when p and q have the same truth values, and false otherwise [9].

2.2.1 Truth Tables for Key Operators

2.2.2 Terminology for $p \rightarrow q$

The implication $p \rightarrow q$ can be expressed in English as [5, 8]:

- "if p , then q "
- " p is sufficient for q " or "a sufficient condition for q is p "
- " p only if q "
- " p implies q "
- " q whenever p "

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \oplus q$	$p \rightarrow q$
0	0	1	0	0	0	1
0	1	1	0	1	1	1
1	0	0	0	1	1	0
1	1	0	1	1	0	1

Table 1: Truth Values of Primary Logical Operators [5, 6, 9]

2.3 Precedence and Associativity

The default precedence of logical operators is: $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$. **UCSC Specific Convention:** In this course, **all operators are right-associative** [10]. For example:

$$p \vee q \vee r \equiv (p \vee (q \vee r))$$

This contrasts with standard arithmetic (where operators like subtraction are left-associative, e.g., $5 - 3 + 2 = (5 - 3) + 2 = 4$) [10].

2.4 Logical Equivalence

When two compound propositions always have the same truth values under all possible assignments, they are called **logically equivalent**, denoted by $\equiv$ or $\Leftrightarrow$ [8]. For $p \rightarrow q$, we define [9]:

- **Converse:** $q \rightarrow p$
- **Contrapositive:** $\neg q \rightarrow \neg p$ (logically equivalent to $p \rightarrow q$)
- **Inverse:** $\neg p \rightarrow \neg q$ (logically equivalent to converse $q \rightarrow p$)

2.4.1 Table of Key Logical Equivalences

- **Identity Laws:** $p \wedge T \equiv p$, $p \vee F \equiv p$ [13]
- **Domination Laws:** $p \vee T \equiv T$, $p \wedge F \equiv F$ [13]
- **Idempotent Laws:** $p \vee p \equiv p$, $p \wedge p \equiv p$ [13]
- **Double Negation Law:** $\neg(\neg p) \equiv p$ [14]
- **Commutative Laws:** $p \vee q \equiv q \vee p$, $p \wedge q \equiv q \wedge p$ [13]
- **Associative Laws:** $(p \vee q) \vee r \equiv p \vee (q \vee r)$, $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$ [13]
- **Distributive Laws:**

$$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$$

$$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$$

[13]

- **De Morgan's Laws:**

$$\neg(p \vee q) \equiv \neg p \wedge \neg q$$

$$\neg(p \wedge q) \equiv \neg p \vee \neg q$$

[13]

- **Absorption Laws:** $p \vee (p \wedge q) \equiv p$, $p \wedge (p \vee q) \equiv p$ [14]
- **Negation Laws:** $p \vee \neg p \equiv T$, $p \wedge \neg p \equiv F$ [14]

2.5 Duality and Functional Completeness

The **dual** of a proposition is formed by interchanging $\wedge$ and $\vee$, and interchanging T and F (or 1 and 0) [14]. The **Principle of Duality** states that if a logical equivalence is true, its dual is also true [14].

A set of logical operators is called **functionally complete** if every compound proposition can be expressed using only those operators [17].

- The set $\{\wedge, \vee, \neg\}$ is functionally complete.
- The set $\{\vee, \neg\}$ is functionally complete since $p \wedge q \equiv \neg(\neg p \vee \neg q)$ [16].
- The set $\{\wedge, \neg\}$ is functionally complete since $p \vee q \equiv \neg(\neg p \wedge \neg q)$.

2.6 Normal Forms

- **Elementary Product:** A conjunction ($\wedge$) of variables and/or their negations (e.g., $p \wedge q \wedge \neg r$) [17].
- **Elementary Sum:** A disjunction ($\vee$) of variables and/or their negations (e.g., $p \vee \neg q \vee r$) [17].
- **Disjunctive Normal Form (DNF):** A sum ($\vee$) of elementary products [18].
- **Conjunctive Normal Form (CNF):** A product ($\wedge$) of elementary sums [18].

2.6.1 Finding DNF and CNF [20, 21]

There are two main methods to find normal forms:

1. **Algebraic Equivalence Method:** Using logical identity laws to rewrite terms.
2. **Truth Table Method:**
 - For **DNF**: Locate the rows where the output is 1 (T). For each such row, form an elementary product that is true only for that row's assignment. Take the disjunction of these products.
 - For **CNF**: Locate the rows where the output is 0 (F). Form an elementary sum that is false only for that row's assignment. Take the conjunction of these sums. Alternatively, find the DNF for $\neg x$, and then negate it using De Morgan's laws.

2.7 Satisfiability and System Specifications

- **Satisfiability:** A proposition is **satisfiable** if there is at least one assignment of truth values to its variables that makes the proposition true [24].
- **Unsatisfiability:** A proposition is **unsatisfiable** (or a contradiction) if there are no assignments that make it true [24, 25].
- **Validity:** A proposition is **valid** (or a tautology) if its output values are all true [25].
- **Consistency:** A set of propositions is **consistent** if and only if there exists an assignment of truth values to all its variables that makes all propositions in the set true *simultaneously* [26].

2.7.1 Example: System Specification Consistency

Determine if these specifications are consistent [27]:

1. "The diagnostic message is stored in the buffer or it is retransmitted." ($p \vee q$)
2. "The diagnostic message is not stored in the buffer." ($\neg p$)
3. "If the diagnostic message is stored in the buffer, then it is retransmitted." ($p \rightarrow q$)

Algebraic Analysis: Assume $\neg p$ is True $\implies p$ is False [28]. Since $p \vee q$ is True and p is False, we must have q is True [28]. Check third specification: $p \rightarrow q \equiv F \rightarrow T \equiv T$ [28]. All three specifications are True when p is False and q is True. Thus, the specifications are **consistent** [28].

3 Lesson 2: Predicate Logic

3.1 Predicates and Propositional Functions

A **predicate** (or propositional function) is an expression containing variables that becomes a proposition once the variables are assigned values [32]. For example, $P(x, y) ::= "x + y = 3"$ is a binary predicate [32]. It is not a proposition because it has no truth value until x and y are instantiated [33]:

- $P(2, 1)$ is True [34].
- $P(3, 4)$ is False [34].

The variable(s) represent the **subject**, and the rest of the statement represents the property (or relation) of the subject [34]. In computer programming, predicates describe program state [35]:

- **Preconditions:** Valid inputs for a program.
- **Postconditions:** Conditions that the output should satisfy after execution.

3.2 Domains and Truth Sets [36]

The set of possible values that variables in a predicate can assume is called the **Domain of Discourse** (or Universe of Discourse), denoted by U . The **Truth Set** of a predicate $P(x)$ over a domain U is the set of all elements $x \in U$ for which $P(x)$ is true [38], written as:

$$\{x \in U \mid P(x) \text{ is true}\}$$

For example, for $P(x) ::= x^2 - 3x + 2 = 0$ over $\mathbb{R}$, the truth set is $\{1, 2\}$ [38, 39].

3.3 Quantifiers

Quantifiers express the extent to which a predicate is true over a domain [39].

1. **Universal Quantifier** ($\forall$): $\forall x P(x)$ means " $P(x)$ is true for all values of x in the domain" [41].
 - If the domain is empty, then $\forall x P(x)$ is vacuously True [42].
 - An element c in the domain for which $P(c)$ is False is called a **counterexample** [41].
2. **Existential Quantifier** ($\exists$): $\exists x P(x)$ means "there exists one or more elements x in the domain for which $P(x)$ is true" [40].

3.4 Binding, Scope, and Logical Equivalences

A variable is **bound** if it is in the scope of a quantifier [22]. Otherwise, it is **free** [22]. For example, in $\exists x(x + y = 1)$, x is bound, and y is free [22]. Important logical equivalences for quantifiers [45]:

$$\forall x(P(x) \wedge Q(x)) \equiv \forall x P(x) \wedge \forall x Q(x)$$

$$\exists x(P(x) \vee Q(x)) \equiv \exists x P(x) \vee \exists x Q(x)$$

Note that $\forall x(P(x) \vee Q(x))$ is **not** logically equivalent to $\forall x P(x) \vee \forall x Q(x)$ [45].

3.4.1 Negation of Quantifiers [46]

$$\neg \forall x P(x) \equiv \exists x \neg P(x)$$

$$\neg \exists x P(x) \equiv \forall x \neg P(x)$$

3.5 Nested Quantifiers and Quantifier Ordering [47]

When multiple quantifiers are nested, the order of quantifiers is critical if they are of different types [48]. Let $K(P, V)$ be "Anti-virus program P kills computer virus V " [48].

- $\exists P \forall V K(P, V)$: "There is an anti-virus program that kills *every* computer virus" [48].
- $\forall V \exists P K(P, V)$: "For *every* computer virus, there is some anti-virus program that kills it" [48].

4 Lesson 3: Arguments and Rules of Inference

4.1 Validity of Arguments

An **argument** in propositional logic is a sequence of propositions. All propositions except the last are called **premises**, and the final proposition is called the **conclusion**. An argument is **valid** if and only if the truth of all its premises implies that the conclusion is true [52]. An **argument form** is a collection of propositions involving propositional variables [52]. It is valid if the conclusion is true whenever the premises are all true [52].

4.2 Rules of Inference for Propositional Logic

Rules of inference are standard valid argument forms used as building blocks for constructing larger proofs [53].

Rule of Inference	Premises & Conclusion	Name
Modus Ponens	$p, p \rightarrow q \quad \therefore q$	Affirming the antecedent [53]
Modus Tollens	$\neg q, p \rightarrow q \quad \therefore \neg p$	Denying the consequent [53]
Hypothetical Syllogism	$p \rightarrow q, q \rightarrow r \quad \therefore p \rightarrow r$	Transitivity of implication [53]
Disjunctive Syllogism	$p \vee q, \neg p \quad \therefore q$	Elimination [53]
Addition	$p \quad \therefore p \vee q$	Disjunction introduction [53]
Simplification	$p \wedge q \quad \therefore p$	Conjunction elimination [53]
Conjunction	$p, q \quad \therefore p \wedge q$	Conjunction introduction [53]
Resolution	$p \vee q, \neg p \vee r \quad \therefore q \vee r$	Resolution rule [53]

Table 2: Propositional Rules of Inference

4.2.1 Logical Fallacies (Invalid Argument Forms)

1. Fallacy of Affirming the Consequence:

$$p \rightarrow q, q \quad \therefore p \quad (\text{Invalid})$$

[59]

2. Fallacy of Denying the Antecedent:

$$p \rightarrow q, \neg p \quad \therefore \neg q \quad (\text{Invalid})$$

[59]

4.3 Rules of Inference for Quantified Statements

- **Universal Instantiation (UI):** $\forall xP(x) \quad \therefore P(c)$ (where c is an arbitrary or specific element).
- **Universal Generalization (UG):** $P(c)$ for an arbitrary $c \quad \therefore \forall xP(x)$.
- **Existential Instantiation (EI):** $\exists xP(x) \quad \therefore P(c)$ (for some specific element c , unknown but fixed) [60].
- **Existential Generalization (EG):** $P(c)$ for some element $c \quad \therefore \exists xP(x)$ [61].

4.4 The Resolution Principle and Refutation Proofs [67]

A **literal** is a variable (p) or its negation ($\neg p$) [66]. A **clause** is a disjunction ($\vee$) of literals [66]. For any two clauses C_1 and C_2 , if there is a literal p in C_1 that is complementary to a literal $\neg p$ in C_2 , we delete both and construct the disjunction of the remaining literals. This constructed clause is the **resolvent** [66].

$$C_1 = p \vee C'_1, \quad C_2 = \neg p \vee C'_2 \quad \therefore C'_1 \vee C'_2$$

[67]

Refutation (Resolution Proof): To prove that a set of clauses is unsatisfiable, we iteratively apply resolution to derive the **empty clause**, denoted by $\square$ or $\emptyset$, which represents a contradiction [67].

5 Lesson 4: Introduction to Proofs

5.1 Terminology of Proofs

- **Proof:** A valid argument that establishes the truth of a mathematical statement [69].
- **Axiom:** A basic assumption accepted as true without proof [70].
- **Theorem:** A statement that can be shown to be true [71].
- **Lemma:** A simple theorem used as an intermediate step to prove a larger theorem [71].
- **Corollary:** A proposition that follows directly from a proven theorem [71].
- **Conjecture:** A statement whose truth value is unknown, but suspected to be true [71]. Once proven, it becomes a theorem [71].

5.2 Methods of Proving Implications $p \rightarrow q$

1. **Direct Proof:** Assume the hypothesis p is true. Use axioms, definitions, and rules of inference to show that q must also be true [73].
2. **Proof by Contraposition** (Indirect Proof): Assume $\neg q$ is true, and show that $\neg p$ must also be true (using direct proof on the contrapositive statement) [69].
3. **Proof by Contradiction:** Assume p is true and q is false (i.e., assume $p \wedge \neg q$ is true). Derive a logical contradiction of the form $r \wedge \neg r$. This shows that the assumption $p \wedge \neg q$ is false, hence $p \rightarrow q$ must be true [69].
4. **Trivial Proof:** Show that the conclusion q is true regardless of p [73].
5. **Vacuous Proof:** Show that the hypothesis p is false [73].

5.3 Proof by Cases and Exhaustive Proofs

To prove $p \rightarrow q$ where $p \equiv p_1 \vee p_2 \vee \dots \vee p_k$, we must prove each subcase separately:

$$(p_1 \vee p_2 \vee \dots \vee p_k) \rightarrow q \equiv (p_1 \rightarrow q) \wedge (p_2 \rightarrow q) \wedge \dots \wedge (p_k \rightarrow q)$$

An **exhaustive proof** is a proof by cases where the number of cases is finite and all possibilities are checked [69].

5.4 Existence and Uniqueness Proofs

- **Existence Proof:** To prove $\exists x P(x)$. Can be **constructive** (finding a specific element c such that $P(c)$ is true) or **non-constructive** [69].
- **Uniqueness Proof:** To prove $\exists! x P(x)$. It requires two steps [74]:
 1. **Existence:** Show that there is an element x with the property $P(x)$ [74].
 2. **Uniqueness:** Show that if $y \neq x$, then $P(y)$ is false; i.e., $P(x) \wedge P(y) \rightarrow x = y$ [74].

6 Lesson 5: Set Theory

6.1 Set Definition and Notation

A **set** is any well-defined collection of distinct objects [76]. Well-defined means there is no ambiguity about whether an object is a member [77].

- $x \in A$ means x is a member of set A [79].
- The number of members in a set A is its **cardinality**, denoted by $|A|$ [79].
- **Null Set (Empty Set)**: The set with no elements, denoted by $\emptyset$ or $\{\}$ [79]. Note that $\{0\}$ is not an empty set [79].
- **Universal Set**: The set of all elements under consideration, denoted by U [79].
- **Power Set**: The set of all subsets of A , denoted by $\mathcal{P}(A)$ [76]. If $|A| = n$, then $|\mathcal{P}(A)| = 2^n$.

6.2 Subsets and Proper Subsets

- A is a **subset** of B (denoted $A \subseteq B$) iff every element of A is also an element of B :

$$A \subseteq B \iff \forall x(x \in A \rightarrow x \in B)$$

[80]

- A is a **proper subset** of B (denoted $A \subset B$ or $A \subsetneq B$) iff $A \subseteq B$ and $A \neq B$ [80]:

$$A \subset B \iff \forall x(x \in A \rightarrow x \in B) \wedge \exists x(x \in B \wedge x \notin A)$$

[80]

6.3 Set Identities

Identity	Alternative Form	Law Name
$A \cap U = A$	$A \cup \emptyset = A$	Identity Laws [81]
$A \cup U = U$	$A \cap \emptyset = \emptyset$	Domination Laws [81]
$A \cup A = A$	$A \cap A = A$	Idempotent Laws [81]
$\overline{\overline{A}} = A$	–	Complementation Law [81]
$A \cup B = B \cup A$	$A \cap B = B \cap A$	Commutative Laws [81]
$(A \cup B) \cup C = A \cup (B \cup C)$	$(A \cap B) \cap C = A \cap (B \cap C)$	Associative Laws [81]
$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$	$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$	Distributive Laws [81]
$\overline{A \cup B} = \overline{A} \cap \overline{B}$	$\overline{A \cap B} = \overline{A} \cup \overline{B}$	De Morgan's Laws [81]
$A \cup (A \cap B) = A$	$A \cap (A \cup B) = A$	Absorption Laws [81]
$A \cup \overline{A} = U$	$A \cap \overline{A} = \emptyset$	Complement Laws [81]

Table 3: Fundamental Set Identities

6.4 Generalized Unions and Intersections [82]

Let $A_1, A_2, \dots, A_n$ be sets.

- **Union**: $\bigcup_{i=1}^n A_i = \{x \mid x \in A_i \text{ for some } i\}$ [82]
- **Intersection**: $\bigcap_{i=1}^n A_i = \{x \mid x \in A_i \text{ for all } i\}$ [82]

7 Lesson 6: Functions

7.1 Formal Definition

Let X and Y be nonempty sets. A **function** f from X to Y (written $f : X \rightarrow Y$) is an assignment of exactly one element of Y to each element of X [84]. If $f(a) = b$, then [85]:

- X is the **domain** of f [85].
- Y is the **codomain** of f [85].
- b is the **image** of a [85].
- a is a **pre-image** of b [85].
- The **range** (or image) of f is the set of all images of elements of X :

$$\text{Range}(f) = \{f(x) \mid x \in X\} \subseteq Y$$

[85]

7.2 Properties of Functions

1. **One-to-One (Injective):** f is injective iff $f(a) = f(b) \implies a = b$ for all a, b in the domain [86].
2. **Onto (Surjective):** f is surjective iff for every $y \in Y$ there exists $x \in X$ such that $f(x) = y$ [87].
3. **One-to-One Correspondence (Bijective):** f is a bijection iff it is both one-to-one and onto [86].

7.3 Inverse Functions and Compositions

- **Inverse Function:** If $f : X \rightarrow Y$ is a bijection, its inverse $f^{-1} : Y \rightarrow X$ is defined by $f^{-1}(y) = x \iff f(x) = y$ [87].
- **Composition:** Let $g : A \rightarrow B$ and $f : B \rightarrow C$. The composition $f \circ g$ is a function from A to C defined by $(f \circ g)(a) = f(g(a))$ [88]. Note that $f \circ g$ is defined only if the range of g is a subset of the domain of f [88].

7.4 Special Functions

- **Floor Function** ($\lfloor x \rfloor$): $\mathbb{R} \rightarrow \mathbb{Z}$ assigns to the real number x the largest integer less than or equal to x [93].
- **Ceiling Function** ($\lceil x \rceil$): $\mathbb{R} \rightarrow \mathbb{Z}$ assigns to the real number x the smallest integer greater than or equal to x [94].
- **Partial Function:** An assignment to each element a in a *subset* of X (the domain of definition) of a unique element b in Y [94]. If the domain of definition equals X , it is a **total function** [94].

8 Lesson 7: Relations

8.1 Cartesian Products and Binary Relations

Let A and B be sets. The **Cartesian Product** of A and B , denoted by $A \times B$, is the set of all ordered pairs (a, b) where $a \in A$ and $b \in B$:

$$A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$$

[95] A **binary relation** R from A to B is a subset of $A \times B$ [95]. We write aRb to denote $(a, b) \in R$ [95].

8.2 Properties of Relations on a Set [95]

A relation R on a set A is a subset of $A \times A$ [95].

1. **Reflexive:** R is reflexive iff $\forall a \in A ((a, a) \in R)$ [95].
2. **Symmetric:** R is symmetric iff $\forall a, b \in A ((a, b) \in R \implies (b, a) \in R)$ [95].
3. **Anti-symmetric:** R is anti-symmetric iff $\forall a, b \in A (((a, b) \in R \wedge (b, a) \in R) \implies a = b)$ [95].
4. **Transitive:** R is transitive iff $\forall a, b, c \in A (((a, b) \in R \wedge (b, c) \in R) \implies (a, c) \in R)$ [95].

8.3 Combining Relations

Relations are sets of ordered pairs, so they can be combined using set operations ($\cup, \cap, -, \oplus$) [96]. The **composite** of relation R (from A to B) and S (from B to C), denoted by $S \circ R$, is:

$$S \circ R = \{(a, c) \mid \exists b \in B ((a, b) \in R \wedge (b, c) \in S)\}$$

[98]

9 Lesson 8: Number Theory and Modular Arithmetic

9.1 Divisibility and Basic Theorems

Let $a, b \in \mathbb{Z}$ with $a \neq 0$. We say a **divides** b (denoted $a \mid b$) if there exists $c \in \mathbb{Z}$ such that $b = ac$ [100]. Here, a is a **factor** or **divisor** of b , and b is a **multiple** of a [100].

- **Theorem:** Let $a, b, c \in \mathbb{Z}$ with $a \neq 0$.
 1. If $a \mid b$ and $a \mid c$, then $a \mid (b + c)$ [101].
 2. If $a \mid b$, then $a \mid bx$ for all $x \in \mathbb{Z}$ [101].
 3. If $a \mid b$ and $b \mid c$, then $a \mid c$ [101].

9.2 Modular Arithmetic and Congruence

If $a, b \in \mathbb{Z}$ and $m \in \mathbb{Z}^+$, then a is **congruent to b modulo m** (written $a \equiv b \pmod{m}$) iff $m \mid (a - b)$ [100]. This is equivalent to stating that $a \bmod m = b \bmod m$ [101].

9.3 Euler's Totient Function $\varphi(n)$

Euler's totient function $\varphi(n)$ is the number of positive integers less than or equal to n that are relatively prime to n ($\gcd(k, n) = 1$) [102]. If $n = p_1^{a_1} p_2^{a_2} \dots p_s^{a_s}$ is the prime power factorization of n , then:

$$\varphi(n) = n \left(1 - \frac{1}{p_1}\right) \left(1 - \frac{1}{p_2}\right) \dots \left(1 - \frac{1}{p_s}\right)$$

[102]

9.4 Residue Systems

- **Complete Residue System modulo n :** A set of n integers $\{0, 1, \dots, n - 1\}$ [103].
- **Reduced Residue System modulo n :** A set of $\varphi(n)$ integers $\{a_1, a_2, \dots, a_{\varphi(n)}\}$ such that $\gcd(a_i, n) = 1$ for all i , and $a_i \not\equiv a_j \pmod{n}$ for $i \neq j$ [102].

9.5 The Euclidean and Extended Euclidean Algorithms

The **Euclidean Algorithm** finds the greatest common divisor $\gcd(a, b)$ of two integers a and b by repeated division. The **Extended Euclidean Algorithm** expresses $\gcd(a, b)$ as a linear combination of a and b :

$$\gcd(a, b) = sa + tb$$

This is crucial for finding **modular multiplicative inverses**. If $\gcd(a, m) = 1$, then $sa + tm = 1 \implies sa \equiv 1 \pmod{m}$, meaning s is the multiplicative inverse of a modulo m .

10 Revision Summary and Quick Reference

10.1 Logical Precedence

Negation ($\neg$) > Conjunction ($\wedge$) > Disjunction ($\vee$) > Conditional ($\rightarrow$) > Bi-conditional ($\leftrightarrow$)

Remember: All operators are right-associative at UCSC!

10.2 Logical Operator vs. Set Operator Translation

Logical Operator	Set Operator	Meaning
Negation ($\neg p$)	Complement ($\bar{A}$)	Not in the set
Conjunction ($p \wedge q$)	Intersection ($A \cap B$)	Element is in both sets
Disjunction ($p \vee q$)	Union ($A \cup B$)	Element is in at least one set
Exclusive Or ($p \oplus q$)	Symmetric Difference ($A \oplus B$)	Element is in exactly one set

Table 4: Translation between Logic and Set Theory [81]

10.3 Proof Strategy Selector

- **Direct Proof:** Best when the definitions can be directly expanded (e.g., "if n is odd, then n^2 is odd").
- **Contraposition:** Best when the negation of the conclusion is simpler to work with than the hypothesis (e.g., "if $3n + 2$ is odd, then n is odd").
- **Contradiction:** Extremely powerful for proving irrationality, infinitude of primes, or showing that a certain configuration is impossible.
- **Cases:** Best when the domain naturally divides into distinct classes (e.g., odd/even integers, positive/negative reals).